Fundamentos de los Sistemas Operativos
Ficha de entrega de práctica

Grupo de prácticas*: **Miembro 1: Romen Adama Caetano Ramirez**	
Número de la práctica*: 1	**Fecha de entrega*:** 14/03/2023

Descripción del trabajo realizado*

Este código implementa una estructura de datos para una sala de cine.

La biblioteca sala.h contiene las constantes usadas por las funciones en este código.
Este código es un archivo de encabezado (header file) llamado "sala.h" que define una serie de funciones y constantes para el manejo de una sala de cine.
Las tres constantes: ERR_NO_CAPACITY, ERR_INVALID_SEAT y ERR_INVALID_SEAT se utilizan para indicar errores en algunas de las funciones definidas.

La estructura de datos Sala tiene dos miembros: asientos, que es un puntero a un array de enteros que representan los asientos, y capacidad, que es la cantidad de asientos en la sala.
Las funciones definidas son:

- *Funcion crea_sala(): reserva memoria para una variable de tipo Sala utilizando malloc(), y asigna el valor de capacidad al miembro capacidad de la estructura Sala. También se reserva memoria para un array de tamaño capacidad de enteros, y se inicializan todos los elementos del array asientos a 0.*
- *Funcion elimina_sala(): libera la memoria reservada para el array asientos y la memoria reservada para la estructura Sala.*
- *Funcion estado_sala(): si la sala ha sido creada, genera el resumen del estado de la sala.*
- *Funcion reserva_asiento(): reserva el primer asiento libre en la sala para un cliente con id, y devuelve el número de asiento reservado. Si no hay asientos libres, devuelve el valor ERR_NO_CAPACITY.*
- *Funcion libera_asiento(): libera un asiento si está ocupado, y devuelve el id del cliente que lo ocupaba. Si el asiento está libre, devuelve -1. Si sala es NULL o si el número de asiento no es válido, también devuelve -1.*
- *Funcion estado_asiento(): devuelve el id del cliente que ocupa un asiento, o 0 si el asiento está libre. Si sala es NULL o si el número de asiento no es válido, devuelve -1.*
- *Funcion asientos_libres(): devuelve la cantidad de asientos libres en la sala. Si sala es NULL, devuelve -1.*
- *Funcion asientos_ocupados() devuelve el número de asientos ocupados en la sala.*
- *Funcion* capacidad() devuelve la capacidad total de la sala.
- La función sentarse() permite a un cliente con un id específico reservar un asiento en la sala.
- *Funcion* levantarse() libera el asiento reservado por un cliente con un id específico.
- *Funcion* reserva_multiple() permite reservar asientos para un grupo de personas. La función recibe como entrada el número de personas en el grupo y una lista de ids de clientes.

El codigo test_sala.c proporciona conjunto de pruebas para la biblioteca "sala.h/sala.c". El archivo incluye las bibliotecas necesarias, define macros para las pruebas y define funciones de prueba.

- *La función test_ReservaBasica prueba la creación de una sala con una capacidad determinada, la reserva de un asiento, la verificación de la capacidad, la comprobación del estado del asiento reservado, la liberación del asiento y la eliminación de la sala.*
- *La función test_EstadoSala prueba el estado de la sala al reservar y liberar asientos.*
- *La función test_SentarseYLevantarse prueba sentarse en un asiento y levantarse de él.*
- *La función test_ReservaMultiple prueba la reserva múltiple de asientos.*
- *La función test_verificar_reservasCapacidad prueba que si la capacidad de la sala se alcanza, no se*

Grupo de prácticas*: **Miembro 1: Romen Adama Caetano Ramirez**	
Número de la práctica*: 1	**Fecha de entrega*:** 14/03/2023

puede reservar otro asiento.

Horas de trabajo invertidas* Miembro 1: 20 horas

Cómo probar el trabajo*

Navegar hasta el directorio donde se encuentre el comprimido y descomprimir, ejemplo:
Unzip Pract1_RACR.zip
Navegar hasta el directorio donde estén los archivos sala.c, sala.h, test_sala.c, sala.o y test_sala.o:
cd /Users/Romen_Adama_CR/Documents/Proyectos/FSO/Practica 1/TareaEntregable/libsala
Comandos para la ejecución del código en mi terminal de MacOS:
//En caso de que el objeto sala.o y test_sala.o no se encuentren ejecutamos previamente
gcc -c sala.c -o sala.o
ar rcs libsalas.a sala.o
gcc -c test_sala.c -o test_sala.o
gcc test_sala.o -L. -lsalas -o test_sala
Esto generar un archivo ejecutable mediante la siguiente orden:
./test_sala

Incidencias

Considero que esta correcto y se realiza lo solicitado en el informe, en caso de incidencia o deficiencia será solventado en la revisión del trabajo.

Comentarios

Si se quiere observar la evolución, el link a continuación es el link de mi GitHub donde he subido mi trabajo, donde se puede ver su desarrollo y además se puede apreciar las tareas de estudio previas que no eran de obligatoriedad desarrollar:
https://github.com/Romen-Adama-Dev/FSO.git

Comentar que el trabajo fue realizado en el IDE Visual Studio Code, y ejecutado mediante la terminal de MacOS, versión del SO Catalina.